

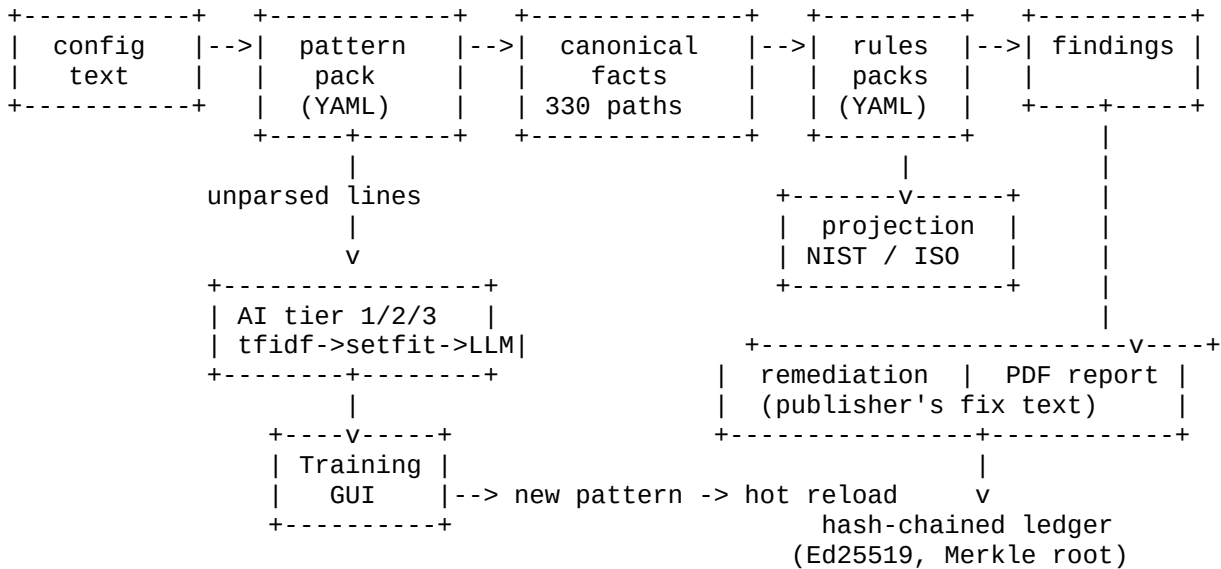
Architecture

PRAMAN — network security compliance auditing. Two pages.

The problem shape

A compliance audit is a join between two things that do not share a vocabulary: a **device configuration** written in a vendor's grammar, and a **benchmark** written in prose about concepts. Every naive design couples them directly — a parser per vendor per benchmark — and then every new vendor multiplies every new framework. Four frameworks and eight vendors is thirty-two parsers.

PRAMAN puts a **canonical fact model** between them, and that single decision determines everything else.



A fact is (path, value, provenance) — for example ("aaa.authentication.login.default", "group tacacs+ local", line 42). Rules are written against paths, never against syntax. So:

- **A new vendor is a pattern pack.** data/ingest/patterns/<vendor>.yaml. The loader globs the directory, so no code changes and no redeploy. (C5)
- **A new benchmark is a mapping pack.** rules/mappings/<pack>/*.yaml, discovered the same way. (C3, C5)
- **The 330 paths are the contract** both sides are tested against.

The five capabilities, and where each one actually lives

C1 — Unified ingestion. backend/ingest/generic.py runs a pattern pack over the text: line patterns, block patterns, and nested blocks. Detection is by content, not by filename. Twenty shipped fixtures across seven vendors parse to 4,302 facts with 46 lines reported unparsed — *reported*, because the unparsed set is the input to C2 and silently dropping a line is how a parser hides its own coverage.

C2 — Training module. Unrecognised lines escalate through three tiers (backend/ai/escalation.py): a TF-IDF classifier, a SetFit few-shot model, then a local quantised LLM. Each tier stops at its confidence threshold and **abstains rather than guessing** — a wrong canonical path is a wrong compliance verdict, which is worse than no verdict. The suggestion goes to the operator in the Training GUI, who confirms or corrects it; the accepted mapping is written to the pattern store and hot-reloaded via RulePackLoader.reload_if_changed(). **No redeploy, no restart** — that is the training requirement, and tests/acceptance/test_training_hot_reload.py is the assertion. With no models installed every tier reports its own absence and the rest of the system is unaffected.

A threshold alone did not deliver that abstention, and the correction is worth recording because the failure mode generalises. Tier 1's calibrated softmax answers $P(class \mid it \text{ is one of my classes})$; it has no way to express "this is not a config line". On an imbalanced 15-class model that became a confident constant — 0.929 for shutdown and 0.929 for zzzzzzzz qqqq alike, above $\tau=0.85$ — so the gate never fired and arbitrary text arrived in the training queue as a 93%-confidence suggestion. The fix compares each prediction against the model's *unconditional* prior and abstains when the input moved it by less than PRIOR_EVIDENCE_MARGIN. Confidence thresholds bound how sure a model claims to be, not whether the question was in its domain; that needs a separate check, and a negative control to find its absence.

C3 — Multi-framework engine. 454 rules over thirteen benchmarks, 42 catalogs and 3,366 normalised controls. Two frameworks are evaluated **directly** against facts (CIS, DISA STIG); NIST SP 800-53 and ISO 27001 are **projected** from them through the publishers' own crosswalks (backend/rules/projection.py) — DISA publishes a CCI -> NIST mapping for every control, so the STIG pack is what lights up all three. Projection is kept separate from evaluation because a projected verdict is weaker evidence than a direct one, and a design that blurred them would let a roll-up look like a measurement.

C4 — Actionable intelligence. Remediation CLI is **extracted from the publisher's own** `fix_text`, never generated (`backend/remediation/engine.py`). `kind` is a tri-state — `cli` when commands were extracted, `manual` when the publisher wrote prose only, `none` when neither source has anything — so a blank block is never returned as if it were a fix. The per-device PDF carries the findings, the severity chart, the fix steps and a `X-PRAMAN-Record-Hash` that ties it to a ledger row.

C5 — Vendor-agnostic scalability. Covered by the canonical model above. The extension point is a directory glob, which is why adding the DISA STIG pack — 34 catalogs, 32 rules, three previously-dark frameworks — required **zero** lines of engine code.

The three decisions worth defending

Honest degradation over graceful failure. Every tier reports its own absence: no signing key means the report is still produced and `X-PRAMAN-Signature` says it is unsigned; no models means the AI tier abstains. The alternative — a system that only works fully configured — cannot be demonstrated on a fresh clone, and a compliance tool that fails closed on a missing optional dependency is a tool nobody runs.

`notchecked` **is a first-class verdict.** A control PRAMAN cannot decide is returned with a reason and **excluded from the score in both directions.** The score is `pass / (pass + fail)`, so coverage can never be improved by declining to check something. All 22 unautomated controls are argued individually in `GAPS.md`. This is the single most important design decision in the project, because the easiest way for a compliance tool to look good is to quietly not check the hard controls.

The ledger is append-only and self-verifying. Each `AuditRecord` binds verdicts to the exact `config_hash` they came from, chains to its predecessor by `prev_hash`, and commits the findings through a Merkle root. `/audit/verify` re-checks four independent links per record — findings, record hash, chain link, signature — so tampering with any earlier record invalidates every record after it. Verification reads rows back from SQLite rather than trusting the process that wrote them.

Stack, and a deliberate deviation

Python 3.10, FastAPI, SQLite, ReportLab. The frontend is **dependency-free ES modules** — 10 views, no build step, no `node_modules`.

The project brief suggested `React/Vite/Tailwind/shadcn/Monaco/xyflow`. That was declined deliberately: the deliverable has to run offline on an air-gapped assessor's laptop from a clone, and a build toolchain is the one dependency that cannot degrade honestly — either `npm install` succeeded on a machine with network access, or there is no UI at all. The cost is hand-written components; the benefit is that `python scripts/serve.py` is the whole deployment story. See `adr/0004-no-frontend-build-step.md`.

Scale and limits

Single-process and SQLite: sized for an estate audited in batches, not for thousands of concurrent uploads. `/ingest/bulk` takes an archive and reports **per-member outcomes**, because an all-or-nothing bulk import never completes on a real estate. Archives are treated as hostile input — member count, per-member size, total expanded size and name traversal are all refused before anything is read. `?background=true` runs the same walk on an in-process job queue and returns a `job_id` to poll or cancel; one worker thread, because WAL takes one writer at a time. A process pool was measured and rejected — see `GAPS.md` §6, where profiling that question found and fixed a 6.9x parse regression.

Identity is per request, and inside the record. Three ordered roles — viewer reads, auditor uploads, approver commits and teaches — resolved from a bearer token on every route except `/health`, `POST /auth/login` and the static frontend. The operator's name is a field of `AuditRecord`, so it is covered by the record hash and the Ed25519 signature: rewriting who ran an audit breaks the same chain that rewriting a verdict breaks. What is still missing is **TLS**, which the application does not implement and will not — see `SECURITY.md` §2 and `../deploy/nginx/praman.conf`.

Every number in this document is measured, not estimated. They come from `reports/metrics/*.json`, regenerated by `scripts/bench/run_all.py` and re-verified against the code by `tests/test_metrics_are_current.py` on every test run. The measured table, including what each figure does *not* support, is in `../README.md#measured-results`.